JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class CountedCompleter<T>

java.lang.Object
    java.util.concurrent.ForkJoinTask<T>
        java.util.concurrent.CountedCompleter<T>

Type Parameters:

T - the type of the result of the completer

All Implemented Interfaces:

Serializable, Future<T>

```
public abstract class CountedCompleter<T>
extends ForkJoinTask<T>
```

A ForkJoinTask with a completion action performed when triggered and there are no remaining pending actions. CountedCompleters are in general more robust in the presence of subtask stalls and blockage than are other forms of ForkJoinTasks, but are less intuitive to program. Uses of CountedCompleter are similar to those of other completion based components (such as CompletionHandler) except that multiple *pending* completions may be necessary to trigger the completion action onCompletion(CountedCompleter), not just one. Unless initialized otherwise, the pending count starts at zero, but may be (atomically) changed using methods setPendingCount(int), addToPendingCount(int), and compareAndSetPendingCount(int, int). Upon invocation of tryComplete(), if the pending action count is nonzero, it is decremented; otherwise, the completion action is performed, and if this completer itself has a completer, the process is continued with its completer. As is the case with related synchronization components such as Phaser and Semaphore, these methods affect only internal counts; they do not establish any further internal bookkeeping. In particular, the identities of pending tasks are not maintained. As illustrated below, you can create subclasses that do record some or all pending tasks or their results when needed. As illustrated below, utility methods supporting customization of completion traversals are also provided. However, because CountedCompleters provide only basic synchronization mechanisms, it may be useful to create further abstract subclasses that maintain linkages, fields, and additional support methods appropriate for a set of related usages.

A concrete CountedCompleter class must define method compute(), that should in most cases (as illustrated below), invoke tryComplete() once before returning. The class may also optionally override method onCompletion(CountedCompleter) to perform an action upon normal completion, and method onExceptionalCompletion(Throwable, CountedCompleter) to perform an action upon any exception.

CountedCompleters most often do not bear results, in which case they are normally declared as `CountedCompleter<Void>`, and will always return `null` as a result value. In other cases, you should override method `getRawResult()` to provide a result from `join(), invoke()`, and related methods. In general, this method should return the value of a field (or a function of one or more fields) of the CountedCompleter object that holds the result upon completion. Method `setRawResult(T)` by default plays no role in CountedCompleters. It is possible, but rarely applicable, to override this method to maintain other objects or fields holding result data.

A CountedCompleter that does not itself have a completer (i.e., one for which `getCompleter()` returns `null`) can be used as a regular ForkJoinTask with this added functionality. However, any completer that in turn has another completer serves only as an internal helper for other computations, so its own task status (as reported in methods such as `ForkJoinTask.isDone()`) is arbitrary; this status changes only upon explicit invocations of `complete(T)`, `ForkJoinTask.cancel(boolean)`, `ForkJoinTask.completeExceptionally(Throwable)` or upon exceptional completion of method `compute`. Upon any exceptional completion, the exception may be relayed to a task's completer (and its completer, and so on), if one exists and it has not otherwise already completed. Similarly, cancelling an internal CountedCompleter has only a local effect on that completer, so is not often useful.

Sample Usages.

Parallel recursive decomposition. CountedCompleters may be arranged in trees similar to those often used with `RecursiveAction`s, although the constructions involved in setting them up typically vary. Here, the completer of each task is its parent in the computation tree. Even though they entail a bit more bookkeeping, CountedCompleters may be better choices when applying a possibly time-consuming operation (that cannot be further subdivided) to each element of an array or collection; especially when the operation takes a significantly different amount of time to complete for some elements than others, either because of intrinsic variation (for example I/O) or auxiliary effects such as garbage collection. Because CountedCompleters provide their own continuations, other tasks need not block waiting to perform them.

For example, here is an initial version of a utility method that uses divide-by-two recursive decomposition to divide work into single pieces (leaf tasks). Even when work is split into individual calls, tree-based techniques are usually preferable to directly forking leaf tasks, because they reduce inter-thread communication and improve load balancing. In the recursive case, the second of each pair of subtasks to finish triggers completion of their parent (because no result combination is performed, the default no-op implementation of method `onCompletion` is not overridden). The utility method sets up the root

task and invokes it (here, implicitly using the `ForkJoinPool.commonPool()`). It is straightforward and reliable (but not optimal) to always set the pending count to the number of child tasks and call `tryComplete()` immediately before returning.

```
public static <E> void forEach(E[] array, Consumer<E> action) {
  class Task extends CountedCompleter<Void> {
    final int lo, hi;
    Task(Task parent, int lo, int hi) {
      super(parent); this.lo = lo; this.hi = hi;
    }

    public void compute() {
      if (hi - lo >= 2) {
        int mid = (lo + hi) >>> 1;
        // must set pending count before fork
        setPendingCount(2);
        new Task(this, mid, hi).fork(); // right child
        new Task(this, lo, mid).fork(); // left child
      }
      else if (hi > lo)
        action.accept(array[lo]);
      tryComplete();
    }
  }
  new Task(null, 0, array.length).invoke();
}
```

This design can be improved by noticing that in the recursive case, the task has nothing to do after forking its right task, so can directly invoke its left task before returning. (This is an analog of tail recursion removal.) Also, when the last action in a task is to fork or invoke a subtask (a "tail call"), the call to `tryComplete()` can be optimized away, at the cost of making the pending count look "off by one".

```
    public void compute() {
      if (hi - lo >= 2) {
        int mid = (lo + hi) >>> 1;
        setPendingCount(1); // looks off by one, but correct!
        new Task(this, mid, hi).fork(); // right child
        new Task(this, lo, mid).compute(); // direct invoke
      } else {
        if (hi > lo)
```

```
          action.accept(array[lo]);
        tryComplete();
      }
    }
```

As a further optimization, notice that the left task need not even exist. Instead of creating a new one, we can continue using the original task, and add a pending count for each fork. Additionally, because no task in this tree implements an `onCompletion(CountedCompleter)` method, `tryComplete` can be replaced with `propagateCompletion()`.

```
    public void compute() {
      int n = hi - lo;
      for (; n >= 2; n /= 2) {
        addToPendingCount(1);
        new Task(this, lo + n/2, lo + n).fork();
      }
      if (n > 0)
        action.accept(array[lo]);
      propagateCompletion();
    }
```

When pending counts can be precomputed, they can be established in the constructor:

```
public static <E> void forEach(E[] array, Consumer<E> action) {
  class Task extends CountedCompleter<Void> {
    final int lo, hi;
    Task(Task parent, int lo, int hi) {
      super(parent, 31 - Integer.numberOfLeadingZeros(hi - lo));
      this.lo = lo; this.hi = hi;
    }

    public void compute() {
      for (int n = hi - lo; n >= 2; n /= 2)
        new Task(this, lo + n/2, lo + n).fork();
      action.accept(array[lo]);
      propagateCompletion();
    }
  }
  if (array.length > 0)
```

```
    new Task(null, 0, array.length).invoke();
}
```

Additional optimizations of such classes might entail specializing classes for leaf steps, subdividing by say, four, instead of two per iteration, and using an adaptive threshold instead of always subdividing down to single elements.

Searching. A tree of CountedCompleters can search for a value or property in different parts of a data structure, and report a result in an `AtomicReference` as soon as one is found. The others can poll the result to avoid unnecessary work. (You could additionally cancel other tasks, but it is usually simpler and more efficient to just let them notice that the result is set and if so skip further processing.) Illustrating again with an array using full partitioning (again, in practice, leaf tasks will almost always process more than one element):

```
class Searcher<E> extends CountedCompleter<E> {
  final E[] array; final AtomicReference<E> result; final int lo, hi;
  Searcher(CountedCompleter<?> p, E[] array, AtomicReference<E> result, int lo, i
    super(p);
    this.array = array; this.result = result; this.lo = lo; this.hi = hi;
  }
  public E getRawResult() { return result.get(); }
  public void compute() { // similar to ForEach version 3
    int l = lo, h = hi;
    while (result.get() == null && h >= l) {
      if (h - l >= 2) {
        int mid = (l + h) >>> 1;
        addToPendingCount(1);
        new Searcher(this, array, result, mid, h).fork();
        h = mid;
      }
      else {
        E x = array[l];
        if (matches(x) && result.compareAndSet(null, x))
          quietlyCompleteRoot(); // root task is now joinable
        break;
      }
    }
    tryComplete(); // normally complete whether or not found
  }
  boolean matches(E e) { ... } // return true if found

  public static <E> E search(E[] array) {
```

```
      return new Searcher<E>(null, array, new AtomicReference<E>(), 0, array.leng
  }
}
```

In
this example, as well as others in which tasks have no other effects except to `compareAndSet` a common result, the trailing unconditional invocation of `tryComplete` could be made conditional (`if (result.get() == null) tryComplete();`) because no further bookkeeping is required to manage completions once the root task completes.

Recording subtasks. CountedCompleter tasks that combine results of multiple subtasks usually need to access these results in method `onCompletion(CountedCompleter)`. As illustrated in the following class (that performs a simplified form of map-reduce where mappings and reductions are all of type `E`), one way to do this in divide and conquer designs is to have each subtask record its sibling, so that it can be accessed in method `onCompletion`. This technique applies to reductions in which the order of combining left and right results does not matter; ordered reductions require explicit left/right designations. Variants of other streamlinings seen in the above examples may also apply.

```
class MyMapper<E> { E apply(E v) {  ...  } }
class MyReducer<E> { E apply(E x, E y) {  ...  } }
class MapReducer<E> extends CountedCompleter<E> {
  final E[] array; final MyMapper<E> mapper;
  final MyReducer<E> reducer; final int lo, hi;
  MapReducer<E> sibling;
  E result;
  MapReducer(CountedCompleter<?> p, E[] array, MyMapper<E> mapper,
             MyReducer<E> reducer, int lo, int hi) {
    super(p);
    this.array = array; this.mapper = mapper;
    this.reducer = reducer; this.lo = lo; this.hi = hi;
  }
  public void compute() {
    if (hi - lo >= 2) {
      int mid = (lo + hi) >>> 1;
      MapReducer<E> left = new MapReducer(this, array, mapper, reducer, lo, mid);
      MapReducer<E> right = new MapReducer(this, array, mapper, reducer, mid, hi)
      left.sibling = right;
      right.sibling = left;
      setPendingCount(1); // only right is pending
      right.fork();
```

```
      left.compute();     // directly execute left
    }
    else {
      if (hi > lo)
          result = mapper.apply(array[lo]);
      tryComplete();
    }
  }
  public void onCompletion(CountedCompleter<?> caller) {
    if (caller != this) {
      MapReducer<E> child = (MapReducer<E>)caller;
      MapReducer<E> sib = child.sibling;
      if (sib == null || sib.result == null)
        result = child.result;
      else
        result = reducer.apply(child.result, sib.result);
    }
  }
  public E getRawResult() { return result; }

  public static <E> E mapReduce(E[] array, MyMapper<E> mapper, MyReducer<E> reduc
    return new MapReducer<E>(null, array, mapper, reducer,
                             0, array.length).invoke();
  }
}
```

Here, method `onCompletion` takes a form common to many completion designs that combine results. This callback-style method is triggered once per task, in either of the two different contexts in which the pending count is, or becomes, zero: (1) by a task itself, if its pending count is zero upon invocation of `tryComplete`, or (2) by any of its subtasks when they complete and decrement the pending count to zero. The `caller` argument distinguishes cases. Most often, when the caller is `this`, no action is necessary. Otherwise the caller argument can be used (usually via a cast) to supply a value (and/or links to other values) to be combined. Assuming proper use of pending counts, the actions inside `onCompletion` occur (once) upon completion of a task and its subtasks. No additional synchronization is required within this method to ensure thread safety of accesses to fields of this task or other completed tasks.

Completion Traversals. If using `onCompletion` to process completions is inapplicable or inconvenient, you can use methods `firstComplete()` and `nextComplete()` to create custom traversals. For example, to define a MapReducer that only splits out right-hand tasks in the form of the third

ForEach example, the completions must cooperatively reduce along unexhausted subtask links, which can be done as follows:

```
class MapReducer<E> extends CountedCompleter<E> { // version 2
  final E[] array; final MyMapper<E> mapper;
  final MyReducer<E> reducer; final int lo, hi;
  MapReducer<E> forks, next; // record subtask forks in list
  E result;
  MapReducer(CountedCompleter<?> p, E[] array, MyMapper<E> mapper,
             MyReducer<E> reducer, int lo, int hi, MapReducer<E> next) {
    super(p);
    this.array = array; this.mapper = mapper;
    this.reducer = reducer; this.lo = lo; this.hi = hi;
    this.next = next;
  }
  public void compute() {
    int l = lo, h = hi;
    while (h - l >= 2) {
      int mid = (l + h) >>> 1;
      addToPendingCount(1);
      (forks = new MapReducer(this, array, mapper, reducer, mid, h, forks)).fork(
      h = mid;
    }
    if (h > l)
      result = mapper.apply(array[l]);
    // process completions by reducing along and advancing subtask links
    for (CountedCompleter<?> c = firstComplete(); c != null; c = c.nextComplete()
      for (MapReducer t = (MapReducer)c, s = t.forks; s != null; s = t.forks = s.
        t.result = reducer.apply(t.result, s.result);
    }
  }
  public E getRawResult() { return result; }

  public static <E> E mapReduce(E[] array, MyMapper<E> mapper, MyReducer<E> reduc
    return new MapReducer<E>(null, array, mapper, reducer,
                             0, array.length, null).invoke();
  }
}
```

Triggers. Some CountedCompleters are themselves never forked, but instead serve as bits of plumbing in other designs; including those in which the completion of one or more async tasks triggers another async task. For example:

```
class HeaderBuilder extends CountedCompleter<...> { ... }
class BodyBuilder extends CountedCompleter<...> { ... }
class PacketSender extends CountedCompleter<...> {
  PacketSender(...) { super(null, 1); ... } // trigger on second completion
  public void compute() { } // never called
  public void onCompletion(CountedCompleter<?> caller) { sendPacket(); }
}
// sample use:
PacketSender p = new PacketSender();
new HeaderBuilder(p, ...).fork();
new BodyBuilder(p, ...).fork();
```

Since:

1.8

See Also:

Serialized Form

Nested Class Summary

Nested classes/interfaces declared in interface Future
Future.State

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	**CountedCompleter**()	Creates a new CountedCompleter with no completer and an initial pending count of zero.
protected	**CountedCompleter**(CountedCompleter<?> completer)	Creates a new CountedCompleter with the given completer and an initial pending count of zero.
protected	**CountedCompleter**(CountedCompleter<?	Creates a new CountedCompleter with

> completer, int initialPendingCount)	the given completer and initial pending count.

Method Summary

All Methods | Instance Methods

Modifier and Type	Method	Description
final void	addToPendingCount(int delta)	Adds (atomically) the given value to the pending count.
final boolean	compareAndSetPendingCoun(int expected, int count)	Sets (atomically) the pending count to the given count only if it currently holds the given expected value.
void	complete(T rawResult)	Regardless of pending count, invokes onCompletion(CountedCompleter), marks this task as complete and further triggers tryComplete() on this task's completer, if one exists.
abstract void	compute()	The main computation performed by this task.
final int	decrementPendingCountUnl	If the pending count is nonzero, (atomically) decrements it.
protected final boolean	exec()	Implements execution conventions for CountedCompleters.
final CountedCompleter<?>	firstComplete()	If this task's pending count is zero, returns this task; otherwise decrements its pending count and returns null.

final CountedCompleter<?>	**getCompleter**()	Returns the completer established in this task's constructor, or `null` if none.
final int	**getPendingCount**()	Returns the current pending count.
T	**getRawResult**()	Returns the result of the computation.
final CountedCompleter<?>	**getRoot**()	Returns the root of the current computation; i.e., this task if it has no completer, else its completer's root.
final void	**helpComplete**(int maxTasks)	If this task has not completed, attempts to process at most the given number of other unprocessed tasks for which this task is on the completion path, if any are known to exist.
final CountedCompleter<?>	**nextComplete**()	If this task does not have a completer, invokes `ForkJoinTask.quietlyComplete()` and returns `null`.
void	**onCompletion**(CountedCompleter<?> caller)	Performs an action when method `tryComplete()` is invoked and the pending count is zero, or when the unconditional method `complete(T)` is invoked.
boolean		

	onExceptionalCompletion(Throwable ex, CountedCompleter<?> caller)	Performs an action when method ForkJoinTask.completeExceptionally(T is invoked or method compute() throws an exception, and this task has not already otherwise completed normally.
final void	**propagateCompletion**()	Equivalent to tryComplete() but does not invoke onCompletion(CountedCompleter) along the completion path: If the pending count is nonzero, decrements the count; otherwise, similarly tries to complete this task's completer, if one exists, else marks this task as complete.
final void	**quietlyCompleteRoot**()	Equivalent to getRoot().quietlyComplete().
final void	**setPendingCount**(int count)	Sets the pending count to the given value.
protected void	**setRawResult**(T t)	A method that result-bearing CountedCompleters may optionally use to help maintain result data.
final void	**tryComplete**()	If the pending count is nonzero, decrements the count; otherwise invokes onCompletion(CountedCompleter) and then similarly tries to complete this task's completer, if one exists,

	else marks this task as complete.

Methods declared in class ForkJoinTask

adapt, adapt, adapt, adaptInterruptible, adaptInterruptible, adaptInterruptible, cancel, compareAndSetForkJoinTaskTag, completeExceptionally, exceptionNow, fork, get, get, getException, getForkJoinTaskTag, getPool, getQueuedTaskCount, getSurplusQueuedTaskCount, helpQuiesce, inForkJoinPool, invoke, invokeAll, invokeAll, invokeAll, isCancelled, isCompletedAbnormally, isCompletedNormally, isDone, join, peekNextLocalTask, pollNextLocalTask, pollSubmission, pollTask, quietlyComplete, quietlyInvoke, quietlyJoin, quietlyJoin, quietlyJoinUninterruptibly, reinitialize, resultNow, setForkJoinTaskTag, state, tryUnfork

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

CountedCompleter

```
protected CountedCompleter(CountedCompleter<?> completer,
                           int initialPendingCount)
```

Creates a new CountedCompleter with the given completer and initial pending count.

Parameters:

`completer` - this task's completer, or `null` if none

`initialPendingCount` - the initial pending count

CountedCompleter

```
protected CountedCompleter(CountedCompleter<?> completer)
```

Creates a new CountedCompleter with the given completer and an initial pending count of zero.

Parameters:

`completer` - this task's completer, or `null` if none

CountedCompleter

```
protected CountedCompleter()
```

Creates a new CountedCompleter with no completer and an initial pending count of zero.

Method Details

compute

```
public abstract void compute()
```

The main computation performed by this task.

onCompletion

```
public void onCompletion(CountedCompleter<?> caller)
```

Performs an action when method `tryComplete()` is invoked and the pending count is zero, or when the unconditional method `complete(T)` is invoked. By default, this method does nothing. You can distinguish cases by checking the identity of the given caller argument. If not equal to `this`, then it is typically a subtask that may contain results (and/or links to other results) to combine.

Parameters:

`caller` - the task invoking this method (which may be this task itself)

onExceptionalCompletion

```
public boolean onExceptionalCompletion(Throwable ex,
                                       CountedCompleter<?> caller)
```

Performs an action when method `ForkJoinTask.completeExceptionally(Throwable)` is invoked or method `compute()` throws an exception, and this task has not already otherwise completed normally. On entry to this method, this task `ForkJoinTask.isCompletedAbnormally()`. The return value of this method

controls further propagation: If `true` and this task has a completer that has not completed, then that completer is also completed exceptionally, with the same exception as this completer. The default implementation of this method does nothing except return `true`.

Parameters:

`ex` - the exception

`caller` - the task invoking this method (which may be this task itself)

Returns:

`true` if this exception should be propagated to this task's completer, if one exists

getCompleter

```
public final CountedCompleter<?> getCompleter()
```

Returns the completer established in this task's constructor, or `null` if none.

Returns:

the completer

getPendingCount

```
public final int getPendingCount()
```

Returns the current pending count.

Returns:

the current pending count

setPendingCount

```
public final void setPendingCount(int count)
```

Sets the pending count to the given value.

Parameters:

`count` - the count

addToPendingCount

```
public final void addToPendingCount(int delta)
```

Adds (atomically) the given value to the pending count.

Parameters:

`delta` - the value to add

compareAndSetPendingCount

```
public final boolean compareAndSetPendingCount(int expected,
                                               int count)
```

Sets (atomically) the pending count to the given count only if it currently holds the given expected value.

Parameters:

`expected` - the expected value

`count` - the new value

Returns:

`true` if successful

decrementPendingCountUnlessZero

```
public final int decrementPendingCountUnlessZero()
```

If the pending count is nonzero, (atomically) decrements it.

Returns:

the initial (undecremented) pending count holding on entry to this method

getRoot

```
public final CountedCompleter<?> getRoot()
```

Returns the root of the current computation; i.e., this task if it has no completer, else its completer's root.

Returns:

the root of the current computation

tryComplete

```
public final void tryComplete()
```

If the pending count is nonzero, decrements the count; otherwise invokes `onCompletion(CountedCompleter)` and then similarly tries to complete this task's completer, if one exists, else marks this task as complete.

propagateCompletion

```
public final void propagateCompletion()
```

Equivalent to `tryComplete()` but does not invoke `onCompletion(CountedCompleter)` along the completion path: If the pending count is nonzero, decrements the count; otherwise, similarly tries to complete

this task's completer, if one exists, else marks this task as complete. This method may be useful in cases where `onCompletion` should not, or need not, be invoked for each completer in a computation.

complete

```
public void complete(T rawResult)
```

Regardless of pending count, invokes `onCompletion(CountedCompleter)`, marks this task as complete and further triggers `tryComplete()` on this task's completer, if one exists. The given rawResult is used as an argument to `setRawResult(T)` before invoking `onCompletion(CountedCompleter)` or marking this task as complete; its value is meaningful only for classes overriding `setRawResult`. This method does not modify the pending count.

This method may be useful when forcing completion as soon as any one (versus all) of several subtask results are obtained. However, in the common (and recommended) case in which `setRawResult` is not overridden, this effect can be obtained more simply using `quietlyCompleteRoot()`.

Overrides:

`complete` in class `ForkJoinTask<T>`

Parameters:

`rawResult` - the raw result

firstComplete

```
public final CountedCompleter<?> firstComplete()
```

If this task's pending count is zero, returns this task; otherwise decrements its pending count and returns `null`. This method is designed to be used with `nextComplete()` in completion traversal loops.

Returns:

this task, if pending count was zero, else `null`

nextComplete

```
public final CountedCompleter<?> nextComplete()
```

If this task does not have a completer, invokes `ForkJoinTask.quietlyComplete()` and returns `null`. Or, if the completer's pending count is non-zero, decrements that pending count and returns `null`. Otherwise, returns the completer. This method can be used as part of a completion traversal loop for homogeneous task hierarchies:

```
for (CountedCompleter<?> c = firstComplete();
     c != null;
     c = c.nextComplete()) {
  // ... process c ...
}
```

Returns:

the completer, or `null` if none

quietlyCompleteRoot

```
public final void quietlyCompleteRoot()
```

Equivalent to `getRoot().quietlyComplete()`.

helpComplete

```
public final void helpComplete(int maxTasks)
```

If this task has not completed, attempts to process at most the given number of other unprocessed tasks for which this task is on the completion path, if any are known to exist.

Parameters:

`maxTasks` - the maximum number of tasks to process. If less than or equal to zero, then no tasks are processed.

exec

```
protected final boolean exec()
```

Implements execution conventions for CountedCompleters.

Specified by:

`exec` in class `ForkJoinTask<T>`

Returns:

`true` if this task is known to have completed normally

getRawResult

```
public T getRawResult()
```

Returns the result of the computation. By default, returns `null`, which is appropriate for `Void` actions, but in other cases should be overridden, almost always to return a field or function of a field that holds the result upon completion.

Specified by:

`getRawResult` in class `ForkJoinTask<T>`

Returns:

the result of the computation

setRawResult

```
protected void setRawResult(T t)
```

A method that result-bearing CountedCompleters may optionally use to help maintain result data. By default, does nothing. Overrides are not recommended. However, if this method is overridden to update existing objects or fields, then it must in general be defined to be thread-safe.

Specified by:

`setRawResult` in class `ForkJoinTask<T>`

Parameters:

`t` - the value

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.